

第 11 讲：Codex / Claude Code: 产品级 Coding Agent Workflow

Coding Agent Notes (June 2026)

本讲在系列中的位置：前十讲已经解释模型、harness、代表系统和人类在环编辑。
已知前提：读者已经知道 agent loop、patch、worktree、权限和 review 的基本含义。
本讲新增：产品级 coding agent 的控制面：surface、workspace、permission、context、evidence、review、background work。
读完后应该能回答：为什么 Codex / Claude Code 的核心不是多一个聊天入口，而是把 agent 行为变成可审查、可恢复、可并行的工程流程。

1 核心图景

产品级 coding agent 和研究型 harness 的差别，不是“功能更多”这么简单。研究型 harness 主要回答一个问题：模型能否在某个环境里完成任务。产品级 workflow 要回答的是另一组问题：任务从哪里进入，在哪个 workspace 运行，哪些动作需要批准，哪些上下文会持久化，失败证据如何保留，最终修改怎样进入 review、commit 或 PR。



这个图可以读成一句话：用户意图先进入某个 session 或 thread，再和项目上下文、记忆、工具配置合并；系统选择 local、worktree 或 cloud workspace；agent 执行动作并产生证据；最后人类通过 diff、日志、测试和 review 决定是否接受。

Codex 和 Claude Code 的具体产品形态不同。Codex 更强调 app、CLI、IDE、cloud、worktrees、review pane、automations 和多 agent 指挥台；Claude Code 更强调 terminal-native 工作流、CLAUDE.md、settings scopes、permission modes、MCP、hooks、background sessions 和跨 surface 延续。但这两条路线背后有同一个系统问题：**agent 的输出必须进入真实开发流程，而真实开发流程要求控制权、证据和可恢复状态。**

Insight. 产品级 coding agent 的关键不是让模型“更自治”，而是让模型的自治被放进可控 workflow：任务在哪里跑、能改哪里、何时需要确认、如何留下证据、怎样进入 review，这些都是能力的一部分。

2 从 Harness 到 Workflow

#05 讲过 harness loop：state、action space、runtime、observation、trajectory、stop、validation。这个抽象对 benchmark 和研究系统很有用，因为它把 agent 的基本闭环讲清楚。但当系统变成产品，loop 外面会长出一圈控制面。

一个实验 harness 可以把失败样本丢掉，可以重建容器，可以只保存最终 patch。产品不行。用户会中途改需求，会问“你为什么删除这个文件”，会希望第二天继续，会要求只接受某个 hunk，会把结果推到 PR，也会担心 agent 看到 secret、运行危险命令或把两个任务的修改混在一起。

层次	主要问题	验收信号
Research harness	能否完成一个任务	patch / tests / score
Agent platform	能否运行、隔离、恢复	state / runtime / permissions
Product workflow	能否进入团队开发流程	diff / evidence / review / PR

这就是为什么 Codex 发布材料会强调 isolated cloud environment、terminal logs、test outputs、AGENTS.md 和人工 review；Claude Code 文档会强调 read-only 默认权限、CLAUDE.md、settings scopes、resume sessions、worktrees、subagents 和脚本化 CLI。这些看起来像产品细节，实际都是把 harness 结果接进真实工程流程的接口。

Workflow 不是 UI 包装。

如果一个 agent 只能在单次对话里给出 patch，它还停留在 assistant 或 benchmark harness 层。只有当它能在开发者已有的 git、terminal、IDE、browser、CI、PR 和 review 流程中留下可解释状态，它才真正进入产品级 workflow。

2.1 一个任务如何穿过 Workflow

把 product coding agent 当成 workflow 系统后，一个任务可以被拆成六个状态。第一步是 intent capture：用户提出目标，但目标通常是不完整的，例如“把登录失败问题修一下”缺少复现步骤、环境、期望行为和验收标准。第二步是 context assembly：系统把当前 repo、打开文件、历史会话、项目规则、记忆、issue、日志和工具配置组合成可用上下文。第三步是 workspace binding：任务必须绑定到 local checkout、worktree、cloud container 或 CI runner，否则“修改”没有明确落点。

第四步是 action execution：agent 读文件、搜索、编辑、运行命令、调用工具。第五步是 evidence capture：系统保存 diff、命令日志、测试输出、失败记录和用户批准历史。第六步是 acceptance：人类通过 review pane、terminal、git diff、PR、CI 或 IDE 结果决定是否接受、部分接受、要求修改或丢弃。

状态	系统要做什么	失败时会怎样
Intent	把自然语言变成任务边界	agent 解决了错误问题
Context	找到项目规则和当前证据	agent 靠猜测补全事实
Workspace	决定修改落点和隔离方式	diff 混入用户未完成工作
Action	执行编辑、命令和工具调用	权限过宽或动作不可复现
Evidence	记录 diff、日志和测试结果	review 只能相信摘要
Acceptance	进入 commit / PR / human review	结果无法合并或无法追责

这个状态机解释了为什么产品文档里很多小功能其实很关键。Resume session 解决的是任务状态延续；worktree 解决的是修改隔离；approval prompt 解决的是动作授权；review pane 解决的是验收对象；AGENTS.md 和 CLAUDE.md 解决的是上下文入口；terminal log 和 test output 解决的是证据。它们不是互相独立的 feature，而是同一条任务生命周期的不同控制点。

Insight. 任务生命周期越长，越不能只依赖最终 patch。长任务必须有中间状态、可恢复上下文、动作授权和验收证据，否则用户在最后看到的只是一个无法信任的大 diff。

2.2 例子：修复一个 CI 失败

假设用户说：“这个 PR 的 CI 挂了，帮我修一下。”如果把它交给普通聊天模型，最常见的结果是模型猜测原因，给出一段看似合理的修改建议。产品级 workflow 的处理方式不同：它先把任务绑定到具体 PR 或 branch，读取 CI 日志和失败测试，再决定是在 local、worktree 还是 cloud 环境里复现。

阶段	Agent 应该产生什么	用户应该检查什么
定位	失败 job、失败命令、相关文件	是否真的看了原始日志
复现	本地或 cloud 中的失败输出	失败是否稳定、环境是否一致
修改	最小 diff 和原因解释	是否越界改动无关模块
验证	修复后命令、输出、剩余失败	是否只验证了 happy path
交付	diff、测试摘要、风险说明	能否进入 PR review

这个例子里，permission 的作用也很具体。读取日志和文件通常可以低风险进行；修改文件需要明确授权；安装依赖、联网、删除文件、重写迁移脚本或推送 branch 都应该进入更强的 approval 或 sandbox 约束。这样用户不是在每个动作上盲目点确认，而是在风险边界变化时重新获得控制权。

Evidence 的作用同样具体。一个合格的 agent 不应该只说“CI failure fixed”。它应该说明：原始失败来自哪个 job，复现命令是什么，失败前输出是什么，修改了哪些 hunk，修复后跑了哪些命令，还有哪些测试没有跑。这样 reviewer 才能判断这个 patch 是真正修复，还是只绕过了可见报错。

这个 walkthrough 也说明 background work 的价值和边界。如果 CI 修复需要等待很长测试，可以放到 worktree 或 cloud 里后台执行；但在合并前，结果仍要回到人类可见的 diff、日志和 PR review。后台执行节省的是等待时间，不应该取消验收。

3 Surface 是控制边界

Terminal、IDE、desktop app、web/cloud、GitHub Action、Slack mention，这些 surface 不只是“在哪里打开 agent”。它们改变了四件事：谁拥有上下文，谁拥有工作区，谁批准动作，以及谁验收结果。

在 terminal 里，agent 靠近开发者当前 shell。它适合本地项目、快速诊断、局部修改和直接运行测试。代价是权限更贴近用户机器，风险也更直接。在 IDE 里，agent 能看到当前文件、selection、diagnostic 和 editor state，适合局部重构和代码理解。desktop app 或 command center 更适合同时管理多个 thread、worktrees 和 long-running tasks。web/cloud surface 则适合把任务 offload 到远程环境，让用户稍后回来 review。

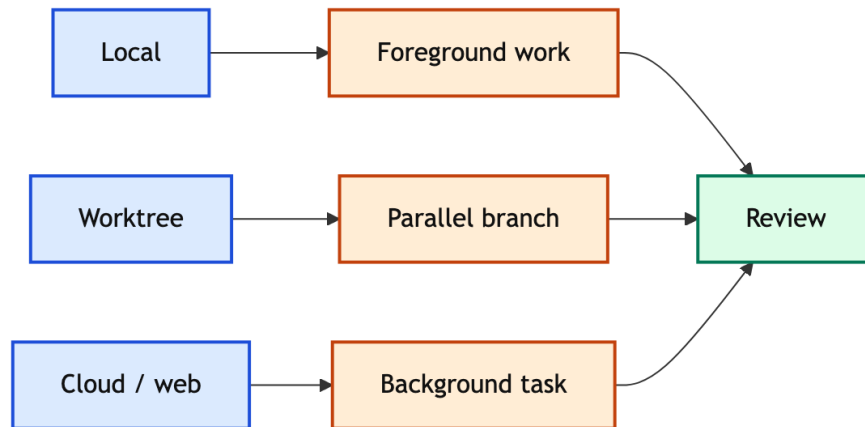
Surface	最强能力	主要风险
Terminal	贴近本地命令和 git	权限过宽、环境副作用
IDE	精准编辑和局部上下文	过度依赖当前 selection
App	多 thread、review、artifact	状态与本地 IDE 分叉
Cloud / Web	背景任务、并行执行	环境 mismatch、证据不足
CI / GitHub	自动 review 和 PR 触发	触发条件、权限和密钥管理

Codex app 文档把 Local、worktree 和 Cloud 明确作为 thread mode；Claude Code overview 则把 terminal、VS Code、desktop app、web、JetBrains、Slack、GitHub Actions 等都放在同一产品族里。两者共同说明：surface 的意义是把同一个 agent 能力放进不同控制边界。

Insight. 评价一个 product coding agent 时，不要只问它支持多少入口。更好的问题是：每个入口如何定义上下文、工作区、权限、证据和验收路径。入口多但边界不清，只会放大不确定性。

4 Workspace: Local、Worktree、Cloud

workspace 是产品级 coding agent 最核心的系统对象之一。它回答“agent 的动作落在哪里”。同一个 prompt，在 local checkout、Git worktree、cloud container 或 CI runner 里运行，含义完全不同。



Local mode 适合 foreground collaboration。开发者正在同一个 checkout 里工作，能马上看 diff、跑服务、切 IDE、补充上下文。这像 Aider 的强项：开发者知道任务方向，agent 负责把编辑执行稳。

worktree mode 适合 parallel branch。Git worktree 让同一个 repo 有多个 checkout，任务可以在后台分支里进行，不污染当前 foreground。Codex manual 明确把 worktree 用于同一项目里的并行任务和 handoff：你可以让 Codex 在后台推进一个想法，等结果成型再切回 local 检查。这里的设计价值不是“复制一份文件”，而是给 agent 一个可隔离、可 review、可丢弃或可合并的工作区。

Cloud 或 web mode 适合 offloaded execution。用户不一定要在本地装依赖，也可以让 agent 在远程环境中跑较长任务。OpenAI 的 Codex 发布页把早期云端任务描述为独立、隔离、预加载 repo 的环境；Claude Code 的 web surface 和 background sessions 也体现类似方向：任务可以离开当前终端持续推进。

Workspace 选择规则：

需要人类即时参与，选 Local；

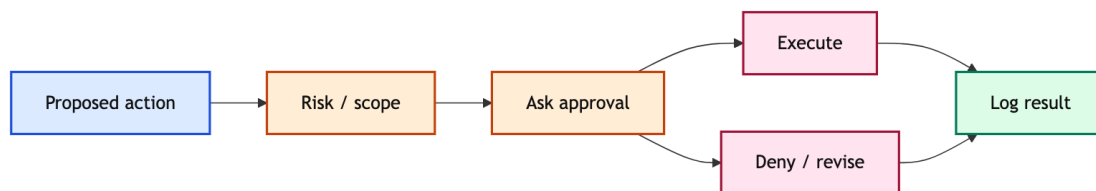
需要并行但仍想保留 git 控制，选 worktree；

需要 offload、长时间运行或没有本地环境，选 Cloud / Web；

需要自动触发，选 CI / GitHub / scheduled workflow。

5 权限和 Sandbox

权限不是安全附属品，而是 workflow 的一部分。因为 product agent 做的不是“回答”，而是会真正读文件、改文件、运行命令、联网、调用工具、创建分支、提交和推送。每一步都可能改变开发状态。



Claude Code security docs 很直接：默认严格 read-only；当需要编辑文件、运行测试或执行命令时，请求显式 permission；用户可以一次性批准，也可以配置自动允许。它还把 sandboxed bash、写入范围限制、prompt fatigue 缓解、Accept Edits mode、网络请求批准、trust verification 等放在安全设计里。

Codex manual 也把 approvals 和 sandbox 分开：approval 决定何时暂停请求许可，sandbox 决定能访问哪些目录和网络。这个区分很重要。Approval 是人类控制点；sandbox 是系统边界。只有 approval 没有 sandbox，用户容易疲劳并误批危险动作；只有 sandbox 没有 approval，系统可能在允许范围内做出用户不想要的修改。

机制	控制什么	常见失败
Read-only default	初始安全边界	agent 无法推进真实修复
Approval prompt	动作是否可执行	prompt fatigue
Sandbox	副作用落点	环境和真实机器不一致
Allowlist / profile	高频安全动作	规则过宽或过旧
Audit trail	事后解释	日志不含决策原因

Insight. 权限系统的目标不是阻止 agent 工作，而是让 agent 的行动权逐步获得、范围清楚、证据可查。好的权限设计会把“能不能做”变成 workflow 中的显式状态，而不是藏在 prompt 里。

6 Context、Memory 和 Instructions

产品级 coding agent 需要长期上下文，但上下文不是越多越好。真正有用的是被正确分层的上下文：项目规则、当前任务、当前文件、历史反馈、工具配置、外部文档和团队偏好，应该有不同来源和不同权重。

Codex 侧的关键是 AGENTS.md、skills、MCP、IDE context 和 project settings。OpenAI 的 Codex 发布页把 AGENTS.md 描述成告诉 Codex 如何导航代码库、运行测试、遵守项目实践的 repo 文档；Codex app 文档还强调 skills、MCP、IDE context、automations 等可以把团队知识和外部工具接入 workflow。

Claude Code 侧的关键是 CLAUDE.md、auto memory、settings scopes、hooks、MCP 和 subagents。Anthropic docs 把 CLAUDE.md 和 auto memory 区分得很清楚：前者是人写的 persistent instructions，后者是 Claude 根据修正和偏好积累的 learnings；两者都会在会话开始时进入上下文，但它们是 context，不是强制配置。真正要阻止动作，需要用 permission、sandbox 或 hook。

上下文类型	典型来源	设计含义
项目规则	AGENTS.md / CLAUDE.md	团队共享约定
个人偏好	user settings / memory	跨项目习惯
当前证据	IDE selection / logs / diff	本轮任务 grounding
外部工具	MCP / plugins / connectors	扩展数据和动作
执行策略	hooks / permissions	不能只靠 prompt

Context 不是 policy。

项目说明可以告诉 agent“应该怎样做”，但不能保证它“一定不会做”。如果某个约束关系到安全、数据、成本或交付质量，就应该进入 permission、sandbox、hook、test 或 review gate，而不是只写进记忆文件。

6.1 上下文分层的实际做法

在真实项目里，上下文管理最容易走向两个极端：一种是把所有东西都塞进 prompt，希望模型自己判断优先级；另一种是只给当前文件，让 agent 缺少系统边界。更稳的做法是把上下文分成四层。

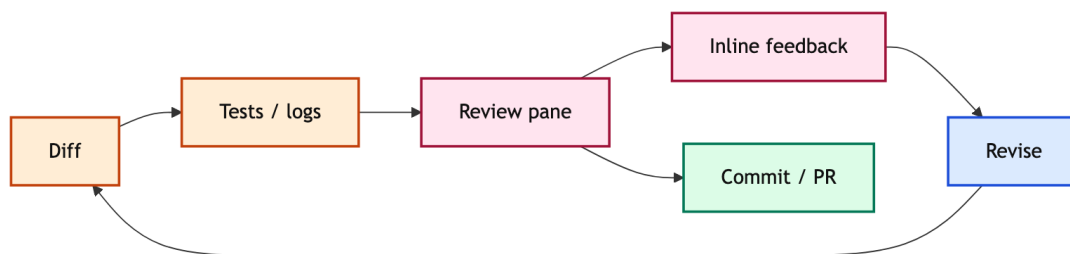
层次	应该放什么	不应该放什么
Project layer	构建命令、测试命令、代码规范、目录约定	临时 debug 结论
Task layer	当前 issue、复现步骤、验收标准、禁止改动范围	与本任务无关的历史偏好
Evidence layer	日志、trace、失败测试、diff、CI 输出	未验证的自然语言猜测
Policy layer	权限、sandbox、hook、review gate	只靠 prompt 表达的安全要求

这四层的权重不同。Project layer 要稳定，适合写进 AGENTS.md、CLAUDE.md 或 repo 文档。Task layer 要短而精确，最好能回答“怎么算完成”。Evidence layer 要贴近当前失败现场，越接近原始日志越好。Policy layer 不能只写成建议，它需要通过权限、hook、CI、review 和组织规则执行。

因此，好的 agent 说明文件不应该像一篇长论文，而应该像项目操作手册：怎样启动服务、怎样跑单测、哪些目录不要碰、怎样格式化、失败时先看哪里、生成代码要遵守什么边界。它越接近开发者日常操作，越能减少 agent 的无效探索。

7 Evidence 和 Review

产品级 agent 的输出不能只是一段“我改好了”。它必须留下 evidence：改了哪些文件，为什么改，运行了哪些命令，哪些测试通过或失败，哪些 action 被批准或拒绝，用户给过哪些 review comment，最终 diff 是否和 summary 一致。



Codex 发布页强调 terminal logs 和 test outputs 是用户验证 agent 行为的证据；Codex manual 的 review pane 进一步把 evidence 接到 Git：它显示 uncommitted changes、branch diff、last turn changes，支持 inline comments、stage、revert 和 PR review context。这个设计把 agent 的修改变成普通工程 review 对象，而不是聊天记录里的承诺。

Claude Code 也有类似路径，只是更 terminal-native：常见 workflow 包括理解代码、定位文件、修 bug、运行测试、创建 pull requests、resume 旧会话、并行 sessions、plan before editing、subagents 和 scripting。CLI reference 还暴露 resume、attach、logs、stop、respawn 等 background-session 操作。这说明 evidence 不只在最后出现，而是贯穿整个会话生命周期。

Insight. Review 的对象不是模型人格，而是可检查证据。一个好的 product coding agent 会把自然语言解释、代码 diff、命令日志、测试结果和用户 feedback 放到同一个验收循环里。

7.1 证据也会失败

证据不是越多越好，而是要能回答 review 的核心问题：这个修改为什么必要，是否真的解决问题，有没有破坏其他行为，哪些部分仍然未知。很多 agent 输出看起来很完整，但证据结构仍然薄弱。

表面上有证据	实际问题	更好的证据
“测试通过” “我修复了 bug”	不知道跑了哪些测试 没有复现失败前状态	命令、输出、覆盖范围、未跑原因 buggy state、失败日志、修复前后对比
“只改了小地方” “按项目规范实现” “可以合并”	diff 仍可能触碰关键路径 没有指向规范来源 没有 review 和 rollback 路径	文件级 diff、hunk 解释、风险点 AGENTS.md / docs / linter / CI 结果 PR、commit、revert strategy、owner sign-off

对团队来说，最有价值的 evidence 是可以被非 agent 工具继续消费的 evidence：Git diff 能被 reviewer 看，terminal log 能被复跑，CI 输出能被链接，PR comment 能被讨论，commit 能被 revert。聊天摘要有帮助，但它不能替代工程对象本身。

8 长任务和并行工作

长期 coding agent 的产品价值，很大一部分来自 background work：你可以让 agent 在另一个 worktree 里写测试、修依赖、做 migration、准备 PR，自己继续处理 foreground 任务。但 background work 只有在边界清楚时才有价值。

并行会带来三个新问题。第一是 task boundary：多个 agent 是否真的在处理可分离任务？如果两个任务都要改同一个核心模块，并行只会制造冲突。第二是 merge boundary：每个 agent 的 diff 能否独立 review、独立撤销、独立测试？第三是 attention boundary：用户什么时候介入，介入时看到什么证据？

并行模式	适合什么	不适合什么
多个 worktrees Background session Subagent research Scheduled task	独立 feature、测试、文档 长时间探索或修复 信息收集、局部调查 周期性 review、CI 分析	同一文件的强耦合修改 需要实时产品判断 共享状态复杂的实现 高风险自动修改

Codex app 把 worktree、cloud、automations 和 thread automations 放在同一个产品系统里；Claude Code docs 则把 background agents、routines、desktop scheduled tasks、/loop、remote control、Slack/GitHub 触发等组织成跨 surface workflow。读这些功能时，不应只看“能不能自动跑”，而要看它们怎样保存状态、怎样隔离修改、怎样把结果带回 review。

9 怎样比较 Codex 和 Claude Code

比较 Codex 和 Claude Code，不要先问“谁功能更多”。更有用的问题是：谁拥有任务状态，谁拥有 workspace，谁承担风险，谁做最终验收。

维度	Codex 倾向	Claude Code 倾向
中心形态 并行方式	app / CLI / IDE / cloud command center worktrees、cloud tasks、automations	terminal-first + app/web/IDE 扩展 background sessions、worktrees、sub-agents
项目规则	AGENTS.md、skills、MCP	CLAUDE.md、auto memory、settings、hooks
权限边界 验收路径	approvals + sandbox profiles review pane、diff、PR、Git tools	read-only default + permission modes terminal/git/PR workflows、CLI logs

这张表不是排名。它说明两类产品都在向同一个方向演化：把 agent 从“能写代码的聊天工具”推进成“能在工程流程中留下可审查状态的协作者”。差别在于产品入口和控制点：Codex 更像多 agent 指挥台，强调从 app 统一管理 local、worktree、cloud、review 和 automation；Claude Code 更像 terminal-native coding operating layer，强调项目记忆、权限、settings、CLI automation 和开发者已有 shell workflow。

正确比较方式：
不是功能清单 vs 功能清单，
而是 surface、workspace、permission、context、evidence、review 六个控制面怎样组合。

10 团队落地检查表

如果一个团队要把 product coding agent 接入日常开发，不应该从“让它自动写更多代码”开始，而应该先决定哪些任务适合 agent、哪些任务必须人类主导、哪些动作可以自动执行、哪些结果必须进入 review。这里的关键是把 agent 当作一个会产生代码变化的开发者，而不是一个只会生成文本的工具。

问题	推荐做法	反模式
任务选择	从测试、文档、局部 bug、低风险迁移开始	直接交给 agent 大范围重构
上下文入口	维护短而准确的 repo 指南	把过时团队知识塞进长 prompt
权限策略	默认只读，逐步开放编辑和命令	一开始就全权限自动执行
工作区	独立任务用 worktree 或 cloud 隔离	多个任务混在同一 checkout
验收	要求 diff、测试、日志和 reviewer sign-off	只看 agent 自述
回滚	保持 commit 边界清楚	把生成结果混进手写修改

一个实用的起步方式是选择“失败成本低、验收信号明确”的任务。例如：给已有函数补测试、修 lint、解释 CI 失败、更新文档、定位简单回归、做小范围 API 迁移。这些任务的好处是目标清楚、diff 可读、测试可跑、失败容易回滚。等团队熟悉权限、上下文和 review 流程后，再逐步开放更长、更模糊、更依赖产品判断的任务。

相反，不适合早期自动化的是那些缺少 oracle 的任务：重新设计核心架构、修改安全边界、调整计费逻辑、迁移生产数据、处理隐私敏感信息、改变跨团队接口。这些任务可以让 agent 做调查、列风险、准备候选 diff，但最终决策和合并必须有 人类 owner。

Insight. 团队采用 coding agent 的成熟度，不取决于它能跑多少自动任务，而取决于它能否把每个自动任务变成边界清楚、证据完整、可 review、可回滚的工程单元。

11 产品工作流验收指标

产品级 agent 的验收不能只看“用户觉得省时间”。更稳定的做法，是把每一次 agent 工作拆成几个可记录指标：任务是否边界清楚，workspace 是否隔离，权限是否按风险触发，证据是否完整，review 是否能快速判断，失败是否能回滚。

指标	应该记录什么	坏味道
Task clarity	输入目标、验收标准、禁止范围	agent 自己猜产品需求
Workspace hygiene	分支、worktree、cloud 环境、dirty diff	自动结果混进人工修改
Permission trace	哪些动作被批准、拒绝或跳过	高风险命令没有记录
Evidence quality	测试、日志、截图、diff rationale	只有“我已经完成”的自述
Review latency	人类能否快速判断是否合并	需要重读完整轨迹
Rollback path	commit / patch / PR 是否可撤销	无法丢弃后台任务结果

这些指标也能帮助比较不同产品形态。一个 terminal-first agent 可能 evidence 很接近 shell log，但跨任务管理弱；一个 app-first agent 可能 review pane 更清楚，但用户需要理解 cloud/worktree 环境差异。真正的产品质量，是这些控制面能否在团队日常流程里稳定闭合。

因此，团队 rollout 时可以先做小规模审计：随机抽取十个 agent 任务，检查每个任务是否有明确输入、隔离 workspace、权限记录、测试证据、review 结论和回滚方式。若这些记录不完整，说明系统还只是“能帮忙写代码”，还没有进入可治理的工程流程。

12 分阶段 Rollout

产品级 coding agent 不适合一次性全量放开。更稳的 rollout 是把任务、权限和验收信号分阶段打开。第一阶段只允许 read-only 调查和解释，让团队观察 agent 是否能找到证据。第二阶段开放低风险 patch，例如文档、测试、lint、局部 bug。第三阶段才开放 background work、并行 worktree 和 PR automation。每一阶段都要有明确退出标准。

阶段	允许能力	进入下一阶段的信号
Read-only	搜索、解释、日志分析、review 建议	evidence 引用准确，误导率低
Low-risk edit	测试、文档、lint、小范围修复	diff 小、测试可复现、回滚清楚
Isolated work	worktree、cloud task、background session	workspace 隔离和 review 流稳定
Workflow automation	PR、scheduled review、CI 辅助	权限、审计、owner 责任清楚

这个阶段化过程能避免两个极端。一个极端是过度保守，只把 agent 当搜索工具，永远得不到自动 patch 的收益。另一个极端是过早授权，让 agent 在没有 review discipline 的情况下修改关键代码。好的 rollout 让能力和治理一起增长：每开放一个新动作，就增加相应 evidence、approval、rollback 和 ownership 规则。

12.1 一次任务审计样例

假设 agent 被要求“修复 checkout 页面偶发 500”。合格的产品工作流不应该只产出一个 diff，而应该留下审计链：任务输入来自哪条 issue；agent 在哪个 worktree 或 cloud 环境运行；它读取了哪些日志和测试；什么时候请求了命令权限；最终跑了哪些验证；哪些路径没有覆盖；reviewer 为什么接受或拒绝 diff。

审计字段	示例
Input	issue link、失败请求、用户约束
Workspace	branch / worktree id、base commit、dirty state
Evidence	error log、复现命令、相关文件、测试输出
Permission	批准了 test command，拒绝了 deploy command
Delivery	diff、commit、PR、review decision、rollback path

这个样例强调的是可追溯性。产品级 agent 的输出不是“我修好了”，而是一组能让团队继续工作的证据包。证据包越标准化，团队越容易把 agent 纳入 code review、CI、incident follow-up 和 knowledge base。

13 常见反模式

第一种反模式是把 agent 当作无限上下文搜索器。用户把问题描述得很模糊，然后让 agent 在整个 repo 里探索。短期看这很方便，长期会让 agent 形成大量不稳定假设。更好的方式是让 agent 先复述任务边界、列出需要验证的证据，再开始改代码。

第二种反模式是把权限问题交给信任感。模型在前几次表现不错，不代表下一次命令安全。权限应该由动作类别、路径、网络、密钥、成本和可回滚性决定，而不是由对模型的主观信任决定。

第三种反模式是把 background work 当成免费并行。并行 agent 会消耗 review 注意力，也会制造 merge 和上下文冲突。只有当任务可拆分、验收标准独立、工作区隔离清楚时，并行才是真的加速。

第四种反模式是让记忆文件变成垃圾场。AGENTS.md、CLAUDE.md 或 auto memory 应该保存稳定规则和高频操作，不应该保存一次性调试结论。否则 agent 会把旧上下文误认为当前事实。

14 局限与开放问题

第一，产品文档变化很快。模型名、权限模式、默认网络策略、云任务能力、价格和可用地区都可能变。讲义里应该保留 workflow mental model，把 volatile 的产品细节放进 source cards 和 claims-to-check，而不是变成正文核心。

第二，权限系统有 prompt fatigue。每一步都问会让用户麻木；问得太少又会让风险被隐藏。真正难的是把低风险高频动作、危险动作、未知动作和组织 policy 分开。

第三，persistent memory 会漂移。记忆可以减少重复解释，也可能保存过时构建命令、局部偏好或错误假设。因此 memory 应该有 audit 和编辑路径，关键约束不能只靠 memory。

第四，worktree 和 cloud 可能制造环境错觉。后台环境跑通，不代表 foreground 本地环境也跑通；远程容器能安装依赖，不代表生产环境或用户机器一样。产品需要把环境差异变成可见证据。

第五，evidence 可能选择性很强。一个 agent 说“测试通过”不够，用户需要知道跑了哪些测试、没跑哪些测试、失败过什么、哪些文件没覆盖。证据越接近 review pane、terminal log、CI 和 PR，越能降低误信风险。

15 最终 takeaway

产品级 coding agent 的核心，不是把一个强模型接到更多按钮上，而是把模型行动放进一个稳定的工程控制面：任务有边界，workspace 有归属，权限有范围，上下文有来源，动作有证据，结果能 review，长任务能恢复，并行任务能隔离。

Codex 和 Claude Code 都说明了同一件事：真正进入开发流程的 agent，不只需要会写代码，还需要把代码变化变成团队可以理解、验证、撤销、提交和继续推进的工作单元。

因此，设计 product coding agent 时，应先问六个问题：这件事从哪个 surface 进入？在哪个 workspace 运行？需要哪些上下文？哪些动作需要批准？完成后有哪些证据？最后由谁 review 并交付？这些问题回答清楚，模型能力才有地方稳定落地。

设计自测:

如果一个任务失败，能否找到原始 evidence?

如果一个后台任务跑偏，能否安全停止、丢弃或回滚?

如果两个 agent 并行修改，能否独立 review 和合并?

如果项目规则变化，agent 的 persistent context 是否会同步更新?

参考资料

- [1] [Codex Manual](#). app modes, worktrees, review, permissions, and workflow behavior
- [2] [Codex Product Page](#). official product positioning and multi-surface framing
- [3] [Introducing Codex](#). cloud task workflow, isolated environments, logs, tests, and review
- [4] [Claude Code Overview](#). surfaces, capabilities, MCP, skills, hooks, and background work
- [5] [Claude Code Security](#). permission architecture, sandboxing, and user responsibility
- [6] [Claude Code Settings](#). configuration scopes, settings files, and permissions
- [7] [Claude Code Memory](#). CLAUDE.md, auto memory, and persistent instructions
- [8] [Claude Code Workflows](#). everyday workflows, resume, worktrees, planning, and subagents
- [9] [Claude Code CLI Reference](#). CLI flags, resume behavior, background agents, logs, and session controls